

UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
“GIOVANNI DEGLI ANTONI”



LAUREA MAGISTRALE IN
SICUREZZA INFORMATICA

TODO: TITOLO DELLA TESI

Autore: Enea Manzi
Matricola: 65935A

Relatore: Prof. Marco Anisetti
Correlatore: Prof. Claudio Agostino Ardagna

A.A. 2025-2026

Abstract

Ringraziamenti

Indice

Abstract	ii
Ringraziamenti	iii
Elenco delle figure	v
Elenco delle tabelle	vi
Elenco dei codici	vii
1 Introduzione	1
1.1 Motivazione e gaps	1
1.2 Obiettivi	1
1.3 Struttura della tesi	1
2 Background e Stato dell'Arte	2
2.1 Esempio di sezione	2
3 Metodologia, Architettura e Principi di Design	3
3.1 Principi Fondamentali: il Perché delle Scelte Architettoniche	3
3.1.1 Agnosticismo Applicativo (D1.P1)	4
3.1.2 La Specifica OpenAPI come Unico Oracolo Formale (D3.P2)	5
3.1.3 Config-Driven Development (D3.P1)	6
3.1.4 Riproducibilità Deterministica (D3.P4)	7
3.2 Architettura dell'Assessment Engine	7
3.2.1 Flusso di Dipendenze Monodirezionale (D1.P2)	8
3.2.2 Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)	8
3.2.3 Split State e Immutabilità (D1.P3)	9
3.2.4 Gateway-Agnostic Adapter (D1.P6)	11
3.2.5 Streaming Evidence Store (D4.P1)	11
3.2.6 Assenza di Dipendenze di Stato Esterno (D3.P5)	12
3.3 Domini di Sicurezza e Box Gradient Applicato	12
3.3.1 Tassonomia degli Otto Domini di Assurance	13
3.3.2 Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)	14

4	Implementazione	16
4.1	La Pipeline di Esecuzione a Sette Fasi e il Core Engine	16
4.1.1	Fail-Safe Error Isolation (D4.P3)	20
4.1.2	Gerarchia delle Eccezioni con Phase Mapping (D4.P4)	20
4.1.3	Costruzione dei Contesti di Esecuzione (D1.P3)	22
4.2	Discovery Dinamico e Risoluzione del Grafo	22
4.2.1	Discovery Dinamico via <code>pkgutil</code> (D2.P1)	23
4.2.2	Scheduling Topologico e <code>DAGScheduler</code> (D2.P2)	24
4.3	Architettura dei Test Nativi e Contratto <code>BaseTest</code>	25
4.3.1	Gerarchia Duale <code>BaseTest</code> / <code>ExternalToolTest</code> (D1.P4)	25
4.3.2	Invarianti di <code>TestResult</code> a Livello di Tipo (D4.P9)	26
4.3.3	Auth Abstraction Layer e Idempotenza dei Token (D2.P5)	27
4.3.4	Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)	28
4.3.5	Oracle a Tre Esiti e Safe HTTP Probing Policy (D4.P7)	28
4.3.6	Path Seed System (D2.P7)	30
4.3.7	Catalogo dei Payload di Attacco (D2.P8)	30
4.3.8	Teardown Best-Effort in Ordine LIFO (D4.P6)	31
4.4	Gerarchia dei Connettori e Integrazione Tool Esterni	31
4.4.1	Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)	32
4.4.2	Lifecycle dei Connector e Dependency Injection (D2.P4)	33
4.4.3	Classificazione A/B dei Connector (D2.P6)	33
4.4.4	Isolamento della Dipendenza AGPL (D7.P2)	34
4.5	Implementazione del Gateway Adapter	34
4.5.1	<code>KongGatewayAdapter</code> : Accesso Read-Only all'Admin API	35
4.5.2	Degradazione Controllata a Tre Livelli (D4.P5)	35
4.6	Evidence Store e Sanitizzazione delle Credenziali	36
4.6.1	Ciclo di Vita dello Streaming Evidence Store (D4.P1)	37
4.6.2	Sanitizzazione Centralizzata delle Credenziali (D4.P2)	37
4.6.3	Dual Audit Trail e Report Domain-Centric (D5.P5)	39
	Bibliografia	41

Elenco delle figure

Figura 4.1 Layout di <code>src/</code> con mapping alle fasi della pipeline.	18
--	----

Elenco delle tabelle

Tabella 3.1	Proprietà APIGuard Assurance per dominio.	4
Tabella 3.2	Domini di Assurance di APIGuard Assurance.	13
Tabella 3.3	Box Gradient applicato in APIGuard Assurance (D3.P3).	15
Tabella 4.1	Fasi della pipeline di assessment (1-2-4 bloccanti; 5-6-7 non-bloccanti).	17
Tabella 4.2	Gerarchia delle eccezioni custom (Fasi 1-4 bloccanti; Fasi 5-6 recupe- rate localmente).	21

Elenco dei codici

Capitolo 1

Introduzione

1.1 Motivazione e gaps

1.2 Obiettivi

1.3 Struttura della tesi

Capitolo 2

Background e Stato dell'Arte

2.1 Esempio di sezione

Come discusso in letteratura [1], ...

Capitolo 3

Metodologia, Architettura e Principi di Design

APIGuard Assurance è un framework di security assessment automatizzato per Application Programming Interface (API) REST esposte tramite gateway, progettato per essere applicazione-agnostico, contract-driven e deterministicamente riproducibile. Nasce dalla necessità di superare la frammentazione degli approcci manuali e target-specifici: strumenti che richiedono di essere riscritti per ogni nuovo target, che producono risultati non confrontabili tra esecuzioni, e che non possono essere integrati in processi di verifica continuativi. Il suo obiettivo è produrre un assessment sistematico, tracciabile e ripetibile, applicabile a qualsiasi API REST documentata senza modifiche al codice.

La presentazione di *APIGuard Assurance* segue un approccio *top-down*: dai principi generali di sistema che ne definiscono il perimetro fino alle scelte implementative che li concretizzano. Architettura e implementazione sono trattate in capitoli distinti perché operano su livelli di astrazione distinti. Il Capitolo 4 mostra come i principi descritti vengono implementati.

Il design di *APIGuard Assurance* è formalizzato in 39 proprietà architetture distribuite su sette categorie; la Tabella 3.1 le raccoglie come mappa di riferimento.¹

3.1 Principi Fondamentali: il Perché delle Scelte Architetture

In coerenza con la progressione *top-down* dichiarata, l'analisi ha inizio dal livello di massima astrazione: le decisioni che definiscono il carattere del tool prima ancora di

¹In grassetto le proprietà trattate in questo capitolo.

Dominio	Proprietà
D1: Architettura & Design	D1.P1, D1.P2, D1.P3 , D1.P4, D1.P5, D1.P6
D2: Estensibilità & Manutenibilità	D2.P1, D2.P2 , D2.P3, D2.P4, D2.P5, D2.P6, D2.P7, D2.P8
D3: Config & Riproducibilità	D3.P1, D3.P2, D3.P3, D3.P4, D3.P5
D4: Robustezza & Sicurezza	D4.P1 , D4.P2, D4.P3, D4.P4, D4.P5, D4.P6, D4.P7, D4.P8, D4.P9
D5: Qualità & Osservabilità	D5.P1, D5.P2, D5.P3, D5.P4, D5.P5
D6: CI/CD & DevEx	D6.P1, D6.P2, D6.P3
D7: Packaging & Deployment	D7.P1, D7.P2, D7.P3

Tabella 3.1: Proprietà APIGuard Assurance per dominio.

descrivere i componenti. Le quattro proprietà discusse in questa sezione costituiscono la struttura portante del sistema: le scelte architetturali trattate nelle sezioni successive si stratificano su di esse, ciascuna risolvendo una frizione specifica all'interno dei vincoli che queste proprietà fondamentali stabiliscono.

3.1.1 Agnosticismo Applicativo (D1.P1)

Quali target applicativi è possibile valutare senza modificare il codice del tool?

Tutta la conoscenza del target viene derivata a runtime da due sole sorgenti: il file `config.yaml`, che raccoglie i parametri operativi del deployment (descritti in dettaglio nella Sezione 3.1.3), e la specifica OpenAPI del target, che fornisce la topologia completa degli endpoint, la definizione dei parametri di path, query e header per ciascuna operazione, gli schemi dei payload di richiesta e risposta, e i codici di stato attesi. Ne consegue un vincolo strutturale preciso: APIGuard Assurance non contiene nessun riferimento hardcoded a path, strutture dati o comportamenti di un'applicazione specifica, e qualsiasi API REST documentata è testabile senza modifiche al codice.

Un test che interroga la superficie di attacco derivata dalla specifica e non individua endpoint pertinenti alle proprie condizioni di esecuzione produce **SKIP**, lo stato previsto per uno scope legittimamente vuoto, distinto dall'**ERROR** che segnalerebbe un malfunzionamento interno al tool. Il principio si estende a qualsiasi requisito non soddisfatto: un tool esterno non installato, la Admin API del gateway non raggiungibile, un insieme di endpoint privi di seed configurato; ciascuna di queste condizioni produce **SKIP** sui test che ne dipendono e lascia proseguire gli altri, a meno che il risultato non costituisca una violazione confermata di un controllo perimetrale fondamentale (come l'assenza totale di autenticazione su tutti gli endpoint esposti), caso in cui la pipeline può essere interrotta esplicitamente tramite configurazione.

La differenza tra uno script target-specifico e un tool di assessment generale non è di complessità, ma di generalità: uno script con path e strutture dati scritti nel codice funziona su quel target e solo su quello (sostituita l'applicazione, va riscritto), mentre un tool che deriva tutta la propria conoscenza dalla specifica formale del target è applicabile a qualsiasi API REST documentata senza modifiche. Questa generalità è precisamente il confine che separa un artefatto di progetto da un contributo riutilizzabile.

Il costo accettato è la dipendenza da una specifica valida. Una specifica obsoleta o deliberatamente incompleta produce un assessment parziale senza che il sistema possa rilevarne l'incoerenza: il tool non ha modo di distinguere un'assenza di vulnerabilità da un'assenza di copertura, perché l'unico oracolo di cui dispone è la specifica stessa. Questa tensione costituisce il limite strutturale discusso nella Sezione ?? e si qualifica come problema aperto nel testing contract-driven, non come difetto correggibile con intervento locale.

3.1.2 La Specifica OpenAPI come Unico Oracolo Formale (D3.P2)

Quale strumento formale governa la costruzione delle richieste e la valutazione delle risposte nell'intera pipeline?

La specifica OpenAPI è l'unico vincolo contrattuale che governa l'intera pipeline: costruisce ogni richiesta nel rispetto dei tipi, dei formati e dei parametri dichiarati, delimita la superficie di attacco agli endpoint documentati e valuta ogni risposta rispetto a ciò che il contratto prevede per quella specifica operazione.

Un fuzzer che genera payload senza conoscere il contratto affronta un ostacolo strutturale prima ancora di raggiungere la logica applicativa: una quota sostanziale delle richieste viene respinta con HTTP 400 dal layer di validazione dell'input, perché i tipi, i formati e i vincoli enumerati nello schema non sono rispettati. Questo è il *combinatorial wall* discusso nel Capitolo 2: lo spazio di tutti i payload sintatticamente possibili è vastissimo, ma lo spazio dei payload strutturalmente validi rispetto al contratto è definito e navigabile. Usare la specifica come oracolo formale abbatte quella barriera: ogni richiesta è costruita nel rispetto dei vincoli dichiarati, spostando il punto di osservazione dalla sintassi alla semantica.

Ne derivano due capacità operativamente distinte rispetto al fuzzing cieco. In primo luogo, la superficie di attacco viene delimitata con precisione: gli endpoint documentati dalla specifica costituiscono l'insieme di ciò che il contratto dichiara raggiungibile, mentre qualsiasi endpoint risponda senza essere incluso in quella lista è, per definizione, una Shadow API. In secondo luogo, le risposte ricevute vengono valutate rispetto a ciò che il contratto prevede per quella richiesta, non rispetto a euristiche generiche: un campo

sensibile restituito in risposta a una richiesta autenticata è un finding solo se lo schema di risposta contrattuale non lo dichiarava atteso.

Anche questa proprietà porta con sé una dipendenza dalla qualità della specifica, già enunciata per D1.P1 (Agnosticismo Applicativo), ma con una conseguenza distinta. In D1.P1 il problema era di copertura: una specifica incompleta riduce gli endpoint osservabili senza che il tool possa rilevarlo. Qui il problema è di affidabilità dei verdeti: una specifica contrattualmente errata produce valutazioni delle risposte basate su un oracolo sbagliato, e il tool non ha modo di distinguere un contratto corretto da uno incorretto. Si tratta del paradosso del Ground Truth, discusso nella Sezione ?? come limite strutturale del paradigma contract-driven e non come anomalia specifica di questa implementazione.

3.1.3 Config-Driven Development (D3.P1)

Esiste un'unica fonte di verità per tutti i parametri che governano il comportamento del tool?

Stabilito che la conoscenza del target risiede nella specifica OpenAPI, rimane da definire il meccanismo che governa l'esecuzione. Tutti i parametri operativi risiedono nel file `config.yaml`, strutturato in aree funzionali con responsabilità distinte: la localizzazione del gateway e della specifica OpenAPI; i parametri di esecuzione globali, tra cui soglia di priorità minima, strategia di filtraggio dei test, timeout per singola esecuzione e formato del logging; il dizionario `path_seed`, che associa a ogni parametro di path parametrico un valore concreto presente sul target; le configurazioni specifiche per singolo test, organizzate per dominio; e i parametri degli strumenti di analisi esterni, con abilitazione per tool, versione attesa e opzioni di invocazione. Le credenziali di autenticazione, per loro natura non versionabili, risiedono in un file `.env` separato, il cui contenuto viene interpolato dal loader prima del parsing. Il codice sorgente non contiene nessun literal decisionale hardcoded: qualsiasi parametro che possa variare tra un'esecuzione e l'altra è dichiarato esplicitamente e validato prima dell'avvio della pipeline.

La separazione tra logica e configurazione assegna a ogni variazione di comportamento un'unica origine dichiarata: il file `config.yaml` attivo durante quella run. Se il comportamento del tool cambia tra un'esecuzione e l'altra, è perché è cambiata la configurazione; se la configurazione è identica, l'output è identico. Questa centralizzazione è anche la condizione che rende possibile la proprietà discussa nella sottosezione seguente: un sistema che legga valori da variabili d'ambiente non dichiarate, da file di stato impliciti o da sorgenti runtime non controllate non può garantire che due esecuzioni successive producano lo stesso risultato, indipendentemente dalla correttezza della logica di test. Fissare la configurazione è il primo passo necessario per fissare l'output.

3.1.4 Riproducibilità Deterministica (D3.P4)

Come viene garantito che l'assessment produca risultati confrontabili tra esecuzioni successive?

Lo stesso target, nella stessa configurazione, interrogato con lo stesso `config.yaml`, deve produrre risultati identici a ogni esecuzione. La riproducibilità è il presupposto che conferisce valore dimostrativo ai risultati del Capitolo ?? : senza di essa, ogni run è un'osservazione singola non confrontabile, e l'assessment non è verificabile da terzi.

Il determinismo è ottenuto attraverso tre meccanismi distinti che si completano a vicenda:

- D3.P1 (Config-Driven Development): se tutti i parametri decisionali sono fissi, la pipeline non ha sorgenti di variazione endogene.
- Struttura del Directed Acyclic Graph (DAG), trattata nella Sezione 3.2.2: un grafo aciclico orientato con risoluzione topologica deterministica produce sempre la stessa sequenza di batch, indipendentemente dall'ordine in cui i test vengono scoperti a runtime.
- Ordinamento lessicografico dei test all'interno di ogni batch del DAG: l'ordinamento topologico puro non garantisce stabilità tra test pronti allo stesso livello, e senza un criterio di tie-breaking esplicito la sequenza potrebbe variare tra esecuzioni; l'ordine lessicografico risolve questa ambiguità, garantendo che la sequenza sia sempre identica a parità di input.

La garanzia di riproducibilità non riguarda ogni componente dell'output: timestamp e identificatori univoci dei singoli record di evidenza variano per natura tra esecuzioni. Ciò che deve essere invariante sono i contatori aggregati (PASS, FAIL, SKIP), i verdetti per singolo test e la distribuzione dei finding per dominio. La validazione empirica, condotta su due run indipendenti che hanno prodotto contatori aggregati byte-equivalenti (9 PASS / 7 FAIL / 2 SKIP / 0 ERROR / 98 finding, con `diff status per-test = 0` e delta wall-clock inferiore allo 0,3%), è discussa nella Sezione ??.

3.2 Architettura dell'Assessment Engine

Le proprietà discusse nella sezione precedente costituiscono la base del sistema. Le scelte di questa sezione si aggiungono ad esse, ciascuna rispondendo a un'esigenza architetturale specifica che le proprietà fondamentali lasciano aperta.

3.2.1 Flusso di Dipendenze Monodirezionale (D1.P2)

Il grafo delle dipendenze tra strati è strettamente aciclico e orientato in una sola direzione: il nucleo del sistema è importato dagli strati di integrazione, che sono importati dagli strati di test, che sono importati dall'orchestratore. Nessun modulo può importare da un livello superiore nella gerarchia. In termini operativi: modificare un test non può rompere il nucleo; aggiungere un connector non può influenzare l'orchestratore; l'intera base del nucleo può essere testata unitariamente senza istanziare alcun client HyperText Transfer Protocol (HTTP) né alcun test di sicurezza.

In assenza di un vincolo esplicito sulla direzione delle dipendenze, la pressione evolutiva naturale del codice tende verso cicli: un test che trova conveniente importare un'utility da un altro test, un connector che ha bisogno di un modello definito nell'orchestratore, e così via. La direzione è unidirezionale: un test può dipendere dal nucleo per accedere ai componenti condivisi, ma nessun modulo del nucleo può dipendere da un test. Ogni ciclo rende un modulo impossibile da isolare, e un modulo non isolabile non è testabile né sostituibile. Il vincolo è verificabile per ispezione del grafo di importazione: nessun modulo dello strato di test importa componenti degli strati superiori, e ogni strato dichiara esplicitamente nel proprio contratto le sole dipendenze che gli sono ammesse. I dettagli di come questo vincolo è garantito a livello di codice sono descritti nella Sezione ??.

La struttura a strati rende praticabile uno degli sviluppi futuri descritti nella Sezione ??: un modulo di test contribuito da terze parti viene scoperto automaticamente² senza che nessuna modifica al nucleo del sistema sia necessaria, perché il nucleo non contiene nessun riferimento ai test che lo utilizzano.

3.2.2 Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)

L'ordine di esecuzione dei test è calcolato a partire dalle dipendenze che ogni test dichiara esplicitamente come parte del proprio contratto. Lo scheduler raccoglie queste dichiarazioni, costruisce un DAG orientato e risolve l'ordine topologico, producendo una sequenza di batch: gruppi di test il cui insieme di dipendenze è già stato soddisfatto. I batch sono sequenziali tra loro; all'interno di ogni batch i test sono privi di dipendenze reciproche. La traduzione di questo schema in codice, incluse le strutture dati e le librerie impiegate, è descritta nella Sezione 4.2.2.

²La scoperta automatica non è priva di vincoli: un modulo viene incluso nella pipeline solo se rispetta i contratti del tool, ovvero se dichiara tutti gli attributi obbligatori e si colloca nella directory attesa. Un modulo che non soddisfi questi requisiti viene ignorato silenziosamente oppure produce un errore di validazione.

Il problema che questa struttura risolve non è solo di ordinamento, ma di correttezza semantica. Un test che verifica la validità crittografica di un token JSON Web Token (JWT) presuppone che il meccanismo di autenticazione funzioni: se eseguito prima del test di autenticazione, il token non è disponibile nel contesto condiviso e il test fallirebbe per la ragione sbagliata. Senza una dichiarazione esplicita della dipendenza, la disponibilità del token è una preconditione implicita che dipende dall'ordine di esecuzione scelto: un cambio di scheduling la viola silenziosamente. Con la dipendenza dichiarata, il DAG verifica la preconditione a monte dell'esecuzione, e un token non disponibile diventa un errore rilevabile prima che un singolo test venga avviato.

Lo scheduler include due proprietà di correttezza strutturale:

- Rilevazione statica dei cicli: se il grafo delle dipendenze contiene un ciclo, l'errore viene segnalato prima che un singolo test venga eseguito, impedendo che stati inconsistenti si propaghino silenziosamente nella pipeline.
- Salvaguardia contro lo stallo: se il grafo risulta ancora attivo ma non riesce a estrarre nessun test pronto, il sistema emette un errore strutturato che elenca i test mai schedulati e interrompe l'esecuzione in modo controllato, fornendo all'operatore la diagnostica necessaria senza dover ricostruire lo stato interno del grafo.

L'ordinamento topologico puro non garantisce un ordine unico tra test pronti allo stesso livello: senza un criterio di tie-breaking³ esplicito la sequenza potrebbe variare tra esecuzioni. Lo scheduler risolve questa ambiguità ordinando lessicograficamente i test all'interno di ogni batch, convertendo la struttura parzialmente ordinata del grafo in una sequenza riproducibile, condizione necessaria per D3.P4 (Reproducibility).

La struttura a batch porta con sé una proprietà che la versione attuale non sfrutta: i test all'interno di uno stesso batch non hanno dipendenze reciproche e sono concettualmente parallelizzabili. Introdurre esecuzione parallela all'interno di un batch è una modifica localizzata allo scheduler, senza impatto sul design dei singoli test, a condizione che le implicazioni di concorrenza sullo stato condiviso vengano analizzate formalmente. Questa traiettoria è documentata come sviluppo futuro nella Sezione ??; la sequenzialità attuale è una scelta deliberata di scope, non un vincolo architetturale.

3.2.3 Split State e Immutabilità (D1.P3)

Durante l'esecuzione sequenziale dei test, due categorie di informazione coesistono nel processo: ciò che si sapeva del target prima che la pipeline iniziasse, e ciò che si scopre o si produce durante l'esecuzione. Trattarle con lo stesso oggetto mutabile introduce

³ *Tie-breaking*: regola di ordinamento applicata tra elementi a parità di livello topologico, per garantire una sequenza stabile e riproducibile tra esecuzioni.

una categoria di bug difficile da diagnosticare: un test che scrive accidentalmente sulla rappresentazione del target altera la vista condivisa da cui ogni verdetto dipende, senza che il sistema possa rilevarlo.

La soluzione è la separazione netta in due oggetti con caratteristiche opposte:

- **TargetContext** - immutabile per costruzione: viene popolato una volta sola prima che la pipeline inizi e qualsiasi tentativo di modifica produce un errore immediato nel punto in cui viene effettuato. Raccoglie tutto ciò che si conosce del target prima dell'esecuzione:
 - la specifica OpenAPI dereferenziata come mappa strutturata di endpoint, parametri e schemi;
 - i parametri di tuning specifici per ogni test ricavati dalla configurazione;
 - l'adapter del gateway iniettato prima dell'avvio della pipeline;
 - le credenziali, risolte dal loader a partire dal file `.env` e congelate nel **TargetContext** tramite la configurazione **frozen**.⁴
- **TestContext** - mutabile, raccoglie ciò che la pipeline produce. L'accesso è strutturato in tre canali tipizzati con responsabilità distinte, anziché in un dizionario ad accesso libero:
 - canale dei token: organizza le credenziali per ruolo, con accesso tramite un identificatore definito dal contratto del sistema, non tramite chiavi stringa arbitrarie;
 - canale di teardown: mantiene un registro Last In, First Out (LIFO) delle risorse create durante la run, garantendo che il cleanup segua l'ordine inverso della creazione, come discusso nella Sezione 4.5.3;
 - canale dei dati condivisi: consente a test in batch successivi di consumare risultati prodotti da batch precedenti, senza che tra i moduli vengano introdotte dipendenze di importazione.

L'implementazione concreta di questa separazione, inclusi i tipi e i meccanismi di accesso, è descritta nella Sezione 4.1.3.

⁴L'accesso avviene esclusivamente tramite i campi tipizzati del modello Pydantic costruito a runtime: non esiste un dizionario libero né accesso diretto ai valori grezzi.

Qualsiasi informazione scoperta durante l'esecuzione (endpoint presenti sul target ma non dichiarati nella specifica, token acquisiti, risorse create) appartiene per definizione allo stato dell'assessment e non alla conoscenza preliminare del target: viene quindi registrata nel contesto di esecuzione, non in quello del target immutabile. La separazione fa sì che la base informativa da cui ogni verdetto dipende non possa essere alterata durante la run, condizione necessaria perché D3.P4 (Reproducibility) sia dimostrabile empiricamente e non solo enunciabile come proprietà attesa.

3.2.4 Gateway-Agnostic Adapter (D1.P6)

D1.P1 stabilisce che la conoscenza del target applicativo deriva esclusivamente dalla specifica OpenAPI e dal `config.yaml`. La proprietà presentata in questa sezione estende lo stesso principio di agnosticismo a un livello distinto, lo strato di controllo del gateway: alcune garanzie di sicurezza, come la verifica che i timeout sui servizi upstream siano configurati, che il plugin di circuit breaking sia attivo o che le credenziali di servizio non siano hardcoded nelle variabili d'ambiente, non sono osservabili interrogando gli endpoint esposti ai client. Queste informazioni risiedono nel piano di configurazione amministrativo del gateway e richiedono un canale di accesso separato dai normali probe HTTP.

Il pattern scelto è un'interfaccia astratta di sola lettura verso il gateway: i test `WHITE_BOX` vi accedono esclusivamente tramite i metodi che questa interfaccia espone, senza contenere nessun riferimento al gateway specifico installato nel deployment. L'implementazione concreta viene selezionata a runtime in base alla configurazione e iniettata nel contesto del target prima che la pipeline abbia inizio. Se la Admin API del gateway non è configurata o non è raggiungibile, tutti i test `WHITE_BOX` transitano a `SKIP` con motivazione esplicita, senza errori a cascata. I dettagli dell'interfaccia e dell'implementazione Kong sono descritti nella Sezione 4.5.1.

Il vantaggio è che aggiungere supporto per un gateway diverso richiede di implementare una nuova classe concreta che soddisfi l'interfaccia, senza toccare nessuno dei test che la utilizzano: la logica di verifica non cambia, cambia solo il meccanismo con cui i dati di configurazione vengono recuperati. Il costo è che l'interfaccia può esprimere solo le verifiche comuni a tutti i gateway supportati: controlli che richiedano funzionalità vendor-specific non generalizzabili rimangono fuori dal perimetro dell'astrazione, e questo costituisce uno dei limiti strutturali discussi nella Sezione ??.

3.2.5 Streaming Evidence Store (D4.P1)

Un finding prodotto da un test acquista valore dimostrabile solo se accompagnato dall'evidenza HTTP che lo supporta: la request inviata, la response ricevuta, il timestamp e

gli identificatori di traccia. Senza di essa, l'esito del test è un'asserzione non verificabile da terzi, priva del requisito di non-repudiabilità che un audit trail tecnico richiede.

Il design originale dell'Evidence Store utilizzava un buffer a capacità fissa in memoria: i record più vecchi venivano rimossi silenziosamente quando il buffer si riempiva, generando riferimenti a record inesistenti nell'audit trail. Il risultato era un trail rotto senza che il sistema producesse nessun errore visibile. Il difetto era strutturale, non correggibile aumentando la capacità: qualsiasi limite fisso avrebbe trasformato la capacità in un parametro di tuning il cui valore corretto dipende da ciò che si vuole misurare, informazione non disponibile a priori.

L'Evidence Store scrive ogni record su file locali con flush immediato, senza buffering in memoria e senza dipendenze verso database o storage remoto: ogni record è permanente nel momento in cui viene prodotto e non esiste un limite superiore al numero di record per test. Il ciclo di vita dettagliato del componente è descritto nella Sezione 4.6.1. La scelta di operare esclusivamente su file locali è anche la condizione che abilita quanto descritto nella sottosezione seguente.

3.2.6 Assenza di Dipendenze di Stato Esterno (D3.P5)

APIGuard Assurance non ha dipendenze di stato esterno a runtime: tutto lo stato mutabile del processo risiede in memoria o in file locali, e le uniche connessioni di rete aperte durante un run sono quelle verso il target applicativo e, opzionalmente, verso la Admin API del gateway. Entrambe sono connessioni di test verso il sistema da valutare, non dipendenze infrastrutturali del tool. Un tool che richiede database, message broker o cache esterna trasferisce la propria affidabilità a servizi esterni: l'exit code al termine dell'esecuzione riflette lo stato dell'assessment solo se tutti quei servizi erano disponibili e corretti, una garanzia che il tool non può fornire autonomamente.

Nessun client di database, cache distribuita o Software Development Kit (SDK) di storage remoto figura tra le dipendenze obbligatorie del progetto: il deployment si riduce all'installazione del package e alla fornitura del file di configurazione. Questa assenza è anche la condizione che rende affidabile D6.P1: i semantic exit codes discussi nella Sezione ?? riflettono fedelmente lo stato dell'assessment perché nessun servizio accessorio può alterarli silenziosamente.

3.3 Domini di Sicurezza e Box Gradient Applicato

Le sezioni precedenti hanno definito le proprietà architetturali e strutturali su cui il sistema si fonda. Questa sezione definisce cosa il sistema misura e secondo quale logica

organizza le proprie misure: è il punto di convergenza tra la tassonomia delle minacce discussa nel Capitolo 2, dove le categorie di vulnerabilità sono state analizzate in quanto problema, e la struttura operativa del tool, dove quelle stesse categorie diventano il perimetro di garanzie che un assessment sistematico è tenuto a coprire.

3.3.1 Tassonomia degli Otto Domini di Assurance

La superficie di sicurezza di un'API esposta tramite gateway non è omogenea. Alcune garanzie sono verificabili senza nessuna conoscenza del sistema, interrogando semplicemente gli endpoint esposti senza credenziali; altre richiedono uno stato autenticato; altre ancora diventano accessibili solo attraverso il piano di configurazione amministrativo del gateway. La tassonomia adottata riconosce questa struttura di precondizioni, organizzando le garanzie in domini che rispecchiano categorie semantiche distinte di rischio.

La Tabella 3.2 li riporta con il loro titolo e l'ambito di verifica che ciascuno racchiude.

Dominio	Titolo	Ambito di verifica
D0	API Discovery	Inventario degli endpoint esposti; rilevazione di Shadow API non documentate nella specifica.
D1	Identity & Authentication	Meccanismi di riconoscimento del chiamante; robustezza del trasporto delle credenziali; ciclo di vita e revoca dei JWT.
D2	Authorization	Logiche di controllo degli accessi; difetti di segregazione orizzontale e verticale (Role-Based Access Control (RBAC), Broken Object Level Authorization (BOLA)).
D3	Data Integrity	Coerenza strutturale dei payload; implementazione delle firme crittografiche (Hash-based Message Authentication Code (HMAC)).
D4	Availability & Resilience	Difese contro l'esaurimento delle risorse; audit di circuit breaker e soglie di rate limiting.
D5	Observability	Qualità e tempestività della telemetria generata dal sistema. (<i>Milestone 2</i>)
D6	Configuration & Hardening	Rafforzamento della configurazione del gateway; policy di routing; header di sicurezza HTTP.
D7	Business Logic & Sensitive Flows	Falle semantiche complesse: elusione dei vincoli di processo, Server-Side Request Forgery (SSRF), race condition su operazioni critiche.

Tabella 3.2: Domini di Assurance di APIGuard Assurance.

La sequenza dei primi tre domini rispecchia una catena logica di precondizioni: non ha senso verificare l'autorizzazione (D2) senza avere prima verificato che l'autenticazione funzioni (D1), così come non ha senso verificare l'autenticazione senza conoscere la superficie di attacco (D0). Un assessment che verifichi D2 in assenza di certezze su D1 raccoglie evidenza su un sistema il cui comportamento in caso di autenticazione compromessa è ignoto. I domini D3-D7 non seguono la stessa dipendenza lineare: appartengono a famiglie semantiche distinte, ciascuna con le proprie precondizioni di accesso, senza vincoli di ordinamento reciproco tra di loro. Questa catena logica si riflette nelle dipendenze dichiarate dai singoli test che operano su questi domini, il cui meccanismo di risoluzione è descritto nella Sezione 3.2.2.

Domain-5 occupa una posizione particolare nella tassonomia: il dominio è architetturealmente predisposto nella struttura del sistema, ma i test corrispondenti sono pianificati per la Milestone 2. La numerazione è riservata intenzionalmente per preservare la coerenza della sequenza nelle versioni successive.

3.3.2 Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)

La classificazione Black/Grey/White Box del testing, nota nella letteratura come distinzione tra gradi di visibilità del sistema sotto test [rif. Sezione 2.X], viene derivata in APIGuard Assurance dalle precondizioni concrete che ogni garanzia impone al tester: non è una categoria assegnata retroattivamente ai test, ma il prodotto di ciò che ciascuna verifica richiede prima ancora di poter iniziare. Alcune garanzie sono fisicamente inaccessibili senza uno specifico livello di privilegio, e quella inaccessibilità è il criterio che determina il livello.

Il mapping adottato, formalizzato come proprietà D3.P3 e sintetizzato nella Tabella 3.3, correla la strategia operativa alle precondizioni concrete che ogni classe di garanzie impone al tester. Il livello di visibilità di ogni test è determinato da ciò che la verifica richiede per essere condotta correttamente.

I controlli perimetrali applicati dal gateway a qualsiasi interlocutore, autenticato o meno, non presuppongono nessuna conoscenza dell'infrastruttura interna: il risultato atteso è lo stesso indipendentemente da chi effettua la richiesta. Eseguirli senza credenziali è l'unico modo per ottenere evidenza non contaminata da privilegi impliciti, ed è per questo che i test `BLACK_BOX` sono la scelta naturale per questa classe di garanzie.

Un controllo di autorizzazione RBAC, al contrario, richiede che il sistema si trovi in uno stato autenticato con almeno due identità di ruolo distinto. Senza quello stato il test non raggiunge nemmeno la logica che intende sondare: il gateway respinge la richiesta al livello di autenticazione, prima ancora che la logica di autorizzazione venga valutata, e il

Strategia	Prerequisiti	Razionale
BLACK_BOX	Nessuno (solo accesso di rete)	Controlli perimetrali applicati dal gateway a qualsiasi interlocutore; simulazione di un attaccante esterno senza credenziali.
GREY_BOX	Token per ≥ 2 ruoli distinti	Garanzie che presuppongono stato autenticato con identità di ruolo distinte; inaccessibili senza credenziali valide.
WHITE_BOX	Accesso Admin API del gateway	Configurazione statica verificabile per ispezione diretta tramite piano di controllo del gateway.

Tabella 3.3: Box Gradient applicato in APIGuard Assurance (D3.P3).

risultato osservato è indistinguibile da un corretto rifiuto di autenticazione. La strategia GREY_BOX è la risposta a questa condizione.

I test WHITE_BOX occupano una posizione metodologicamente distinta. Alcune garanzie sono più efficacemente verificabili tramite ispezione diretta della configurazione del gateway che tramite probe empirici verso gli endpoint: la versione Transport Layer Security (TLS) supportata, la presenza e la parametrizzazione dei plugin di sicurezza, l'assenza di credenziali hardcoded nelle variabili d'ambiente. Un test empirico che cerchi di inferire questi parametri dall'esterno richiederebbe attacchi elaborati con esito incerto; un audit della configurazione li verifica in modo deterministico e riproducibile. È per questo che i test WHITE_BOX delegano la lettura della configurazione all'adapter del gateway descritto nella Sezione 3.2.4, anziché effettuare probe sull'interfaccia pubblica.

In contesti industriali, la disponibilità delle credenziali e dell'accesso amministrativo varia da deployment a deployment. Un'organizzazione che non dispone di accesso amministrativo al gateway può eseguire l'intera suite BLACK_BOX e GREY_BOX, mentre i test WHITE_BOX transitano a SKIP con motivazione esplicita. La Sezione ?? descrive come questa variabilità di privilegio si integri con le pipeline Continuous Integration / Continuous Deployment (CI/CD).

Capitolo 4

Implementazione

Le proprietà architetturali formalizzate nel Capitolo 3 si manifestano in questa sezione nella loro implementazione concreta. Il grado di trattamento rispecchia il peso strutturale di ciascuna: alcune definiscono l'organizzazione di interi componenti e sono trattate in sezioni dedicate; altre derivano dalle proprietà stabilite nel Capitolo 3 e vengono richiamate nel contesto del componente in cui si realizzano; altre ancora, di natura prevalentemente operativa, trovano collocazione nelle sezioni di questo capitolo pertinenti al loro locus, o nei capitoli successivi dedicati alla validazione e all'integrazione con i processi di sviluppo. La Tabella 3.1 costituisce la mappa di navigazione verso le sezioni dove ciascuna proprietà è introdotta.

Il Capitolo 3 ha percorso il primo livello di questo approccio top-down: ha stabilito i principi fondamentali del sistema, la metodologia di assessment, i domini di sicurezza e le proprietà architetturali che ne definiscono la struttura. Questo capitolo ne rappresenta il livello successivo nella discesa: i principi prendono forma in moduli Python, le proprietà si concretizzano in classi, meccanismi e scelte implementative specifiche. Dove un componente è stato introdotto nella sua motivazione architetturale, le sezioni seguenti aggiungono il dettaglio implementativo con un rimando esplicito alla sezione di origine.

4.1 La Pipeline di Esecuzione a Sette Fasi e il Core Engine

L'intero assessment si svolge in sette fasi sequenziali, orchestrate da `engine.py`. La sequenza rispecchia una dipendenza logica reale tra le operazioni, dove ogni fase produce le precondizioni che la successiva presuppone; tale dipendenza determina anche la semantica degli errori. Le fasi 1, 2 e 4 producono la configurazione validata, la mappa degli endpoint e il grafo delle dipendenze tra test: senza questi tre elementi nessun test può essere eseguito in modo significativo, e un errore in una qualsiasi di esse rende l'in-

tero assessment privo di fondamento. Le fasi 5, 6 e 7 operano invece su un contesto già validato, dove il fallimento è sempre recuperabile localmente: un test fallito produce un `TestResult(ERROR)` e la pipeline prosegue; un teardown parziale viene registrato come `WARNING` senza invalidare i risultati che lo precedono.

Fase	Cartella	File
1. Inizializzazione	<code>config/</code>	<code>loader.py</code>
2. OpenAPI Discovery	<code>discovery/</code>	<code>openapi.py</code>
3. Costruzione Contesti	<code>src/</code>	<code>engine.py</code>
4. Test Discovery e DAG	<code>tests/</code> , <code>core/</code>	<code>registry.py</code> , <code>dag.py</code>
5. Esecuzione	<code>src/</code>	<code>engine.py</code>
6. Teardown	<code>core/</code>	<code>context.py</code>
7. Report	<code>report/</code>	<code>renderer.py</code>

Tabella 4.1: Fasi della pipeline di assessment (1-2-4 bloccanti; 5-6-7 non-bloccanti).

La Tabella 4.1 riporta il mapping tra fase e modulo responsabile: ogni fase corrisponde a un modulo separato nel filesystem (Figura 4.1), e `engine.py` è il solo file autorizzato a importare da tutti gli strati (Sezione 3.2.1). Il dettaglio di ogni fase viene presentato nell'elenco sottostante; si nota che le eccezioni citate per le fasi bloccanti appartengono alla gerarchia custom analizzata nella Sezione 4.1.2, dove il loro phase mapping e le relative politiche di recovery sono descritte.

1. **Fase 1: Inizializzazione** (Sezione 3.1.3). `config/loader.py` legge e valida `config.yaml` contro lo schema Pydantic, interpola i segreti da variabili d'ambiente e costruisce il modello `RuntimeConfig`.
 - *In caso di errore:* `ConfigurationError` interrompe lo startup.
 - *Perché bloccante:* senza configurazione valida non esistono credenziali, Uniform Resource Locator (URL) di target né parametri di esecuzione; qualsiasi fase successiva opererebbe su basi non definite.
2. **Fase 2: OpenAPI Discovery** (Sezione 3.1.2). `discovery/openapi.py` scarica la specifica OpenAPI del target, dereferenzia i `$ref`¹ tramite `prance`, la valida con `openapi-spec-validator` e costruisce l'`AttackSurface`: la mappa strutturata degli endpoint che ogni test interroga per determinare il proprio scope.
 - *In caso di errore:* `OpenAPILoadError` interrompe lo startup.

¹Le specifiche OpenAPI usano `$ref` per referenziare schemi riutilizzabili definiti altrove nel documento o in file esterni. La dereferenziazione sostituisce ogni `$ref` con il contenuto del componente referenziato, producendo una specifica autonoma e navigabile senza dipendenze esterne.

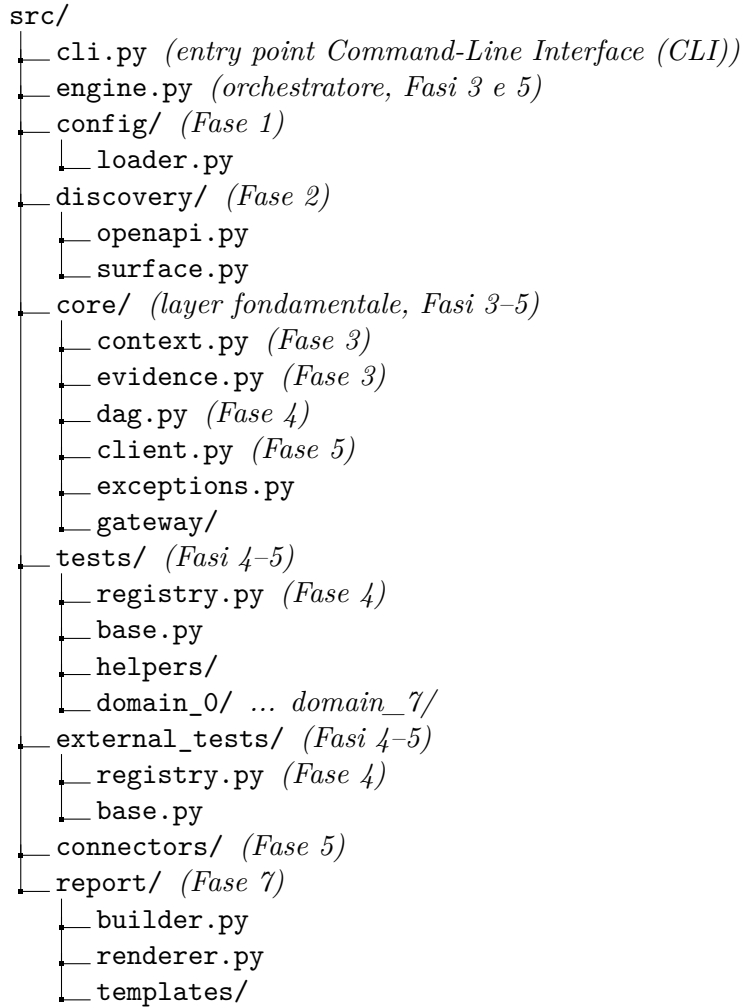


Figura 4.1: Layout di `src/` con mapping alle fasi della pipeline.

- *Perché bloccante:* la specifica OpenAPI è la sorgente esclusiva della conoscenza del target (Sezione 3.1.1); senza di essa nessun test può determinare su quali endpoint operare.

3. **Fase 3: Costruzione Contesti.** `engine.py` istanzia i tre oggetti di runtime condivisi: `TargetContext` (conoscenza del target frozen) e `TestContext` (stato dell'assessment mutabile), descritti nella Sezione 4.1.3, e `EvidenceStore` (streaming su disco), descritto nella Sezione 4.6.1.

- *In caso di errore:* nessuna eccezione prevista; la costruzione dei contesti non produce fallimenti recuperabili.

4. **Fase 4: Test Discovery e DAG.** `tests/registry.py` scopre dinamicamente i test disponibili via `pkgutil` e applica i filtri di proprietà e strategia configurati (Sezione 4.2.1); la lista risultante viene passata a `core/dag.py`, che costruisce il grafo delle dipendenze e calcola l'ordinamento topologico in batch (Sezione 4.2.2).
 - *In caso di errore:* `DAGCycleError` interrompe lo startup.
 - *Perché bloccante:* un ciclo nelle dipendenze rende il grafo non risolvibile; tentare l'esecuzione produrrebbe stallo o ordinamento non deterministico.
5. **Fase 5: Esecuzione.** `engine.py` itera sui `ScheduledBatch` prodotti dalla fase precedente ed esegue i test in ordine topologico, invocando `BaseTest.execute()` per i test nativi (Sezione 4.3) e `ExternalToolTest.execute()` per i test con tool esterni (Sezione 4.4); il meccanismo di isolamento degli errori è descritto nella Sezione 4.1.1.
 - *In caso di errore:* le eccezioni interne a ogni test sono catturate dal contratto di `execute()` e convertite in `TestResult(ERROR)`; la pipeline prosegue con il test successivo.
 - *Perché non-bloccante:* il fallimento di un singolo test è un dato dell'assessment, non un motivo per invalidare i risultati degli altri.
6. **Fase 6: Teardown.** `core/context.py` drena il registro delle risorse create durante l'esecuzione e tenta la loro rimozione in ordine LIFO tramite `SecurityClient`; il meccanismo è descritto nella Sezione 4.3.8.
 - *In caso di errore:* `TeardownError` registrato come `WARNING` strutturato; non invalida i risultati dell'assessment.
 - *Perché non-bloccante:* il mancato cleanup è un avvertimento operativo, non un'invalidazione scientifica dei finding già prodotti.
7. **Fase 7: Report.** `report/builder.py` aggrega i `TestResult` in un `ReportData` e produce tre output: `evidence.json` (archivio forense), `assessment_report.html` (report operativo con log completo delle transazioni) e `apiguard_report.json` (serializzazione strutturata per CI/CD), descritti nella Sezione 4.6.3.
 - *In caso di errore:* ogni sotto-operazione di 7 dispone di un proprio blocco di gestione indipendente; un fallimento su un output non impedisce la produzione degli altri.

Il ruolo di `engine.py` è limitato all'orchestrazione pura: esegue le fasi nell'ordine corretto, trasferisce i contesti da una fase all'altra, e non contiene logica di dominio di nessun tipo (né regole di sicurezza, né criteri di valutazione, né conoscenza del formato della specifica OpenAPI). La scelta ha una ragione strutturale: un orchestratore che incorpora logica di dominio diventa il punto di concentrazione delle dipendenze, rendendo difficile testare i componenti in isolamento e trasformando ogni sostituzione di fase in un'operazione ad alto rischio di regressione. La coerenza con il principio del flusso monodirezionale (Sezione 3.2.1) è verificabile per ispezione diretta: `engine.py` importa da `core/`, `discovery/`, `tests/` e `report/`, ma nessuno di questi moduli importa da `engine.py`.

4.1.1 Fail-Safe Error Isolation (D4.P3)

Il contratto tra l'orchestratore (`engine.py`) e i test individuali si caratterizza per un'assenza strutturale: nessun blocco `try/except` racchiude la chiamata a `test.execute()` nell'engine. Si tratta di una scelta deliberata, che delega la responsabilità della gestione delle eccezioni a livello di test, impedendo che una singola probe possa interrompere l'esecuzione delle altre per errori non previsti. Il perimetro di responsabilità dell'engine termina alla firma del contratto: invoca `test.execute()` e riceve un `TestResult`; qualsiasi condizione interna al test, normale o anomala, non attraversa quel confine.

Perché questo contratto funzioni, `BaseTest.execute()` deve garantire due invarianti: restituire sempre un `TestResult` e non propagare mai eccezioni verso l'esterno. La realizzazione concreta è un blocco `try/except` `Exception` all'interno di `execute()`: in caso di eccezione imprevista, l'helper `_make_error()` costruisce un `TestResult(status=ERROR, message=str(e))` con il traceback troncato a 500 caratteri. Un test con un bug interno, una risposta HTTP inattesa o un timeout non gestito produce un `TestResult(ERROR)` registrato nell'evidenza con il proprio traceback; la pipeline prosegue con il test successivo nel batch senza interruzioni. La solidità complessiva del sistema non è condizionata dalla correttezza di nessun singolo test.

4.1.2 Gerarchia delle Eccezioni con Phase Mapping (D4.P4)

Il meccanismo che rende possibile la distinzione tra errori bloccanti e non-bloccanti descritto in Tabella 4.1 è la gerarchia delle eccezioni custom. Tutte le eccezioni del tool ereditano da `ToolBaseError`, radice comune della gerarchia. La struttura offre al codice chiamante due granularità di intercettazione: un `except ToolBaseError` copre qualsiasi condizione di errore del tool in un unico punto; scendere su un'eccezione specifica, come `except ConfigurationError` o `except DAGCycleError`, permette di distinguere la causa e rispondere in modo differenziato. Tale granularità rispecchia in modo diretto la distinzione tra errori bloccanti e non-bloccanti stabilita a livello architetturale, con-

cretizzandosi in 11 classi, distribuite su 3 file, riportate nella Tabella 4.2 con la relativa fase.

Classe	File	Comportamento
<code>ToolBaseError</code>	<code>exceptions.py</code>	Radice della gerarchia; non si solleva direttamente
<code>ConfigurationError</code>	<code>exceptions.py</code>	Fase 1: fatale, config non valida; blocca lo startup
<code>OpenAPILoadError</code>	<code>exceptions.py</code>	Fase 2: fatale, spec irraggiungibile o invalida; blocca lo startup
<code>DAGCycleError</code>	<code>exceptions.py</code>	Fase 4: fatale, ciclo nelle dipendenze; blocca lo startup
<code>AuthenticationSetupError</code>	<code>exceptions.py</code>	Helper auth: fatale se le credenziali configurate sono invalide
<code>SecurityClientError</code>	<code>exceptions.py</code>	Fase 5: recuperata, produce <code>TestResult(ERROR)</code>
<code>ExternalToolError</code>	<code>exceptions.py</code>	Fase 5: recuperata, produce <code>TestResult(ERROR)</code>
<code>TeardownError</code>	<code>exceptions.py</code>	Fase 6: recuperata, registrata come <code>WARNING</code>
<code>GatewayAdapterError</code>	<code>gateway/base.py</code>	Fase 5: recuperata, produce <code>TestResult(ERROR)</code>
<code>SeedGeneratorFetchError</code>	<code>seed_generator.py</code>	CLI <code>generate-seed</code> : catturata in <code>cli.py</code> , exit code 10
<code>SeedGeneratorParseError</code>	<code>seed_generator.py</code>	CLI <code>generate-seed</code> : catturata in <code>cli.py</code> , exit code 10

Tabella 4.2: Gerarchia delle eccezioni custom (Fasi 1–4 bloccanti; Fasi 5–6 recuperate localmente).

La prima scelta che riflette una decisione di design consapevole e non un vincolo tecnico, riguarda la collocazione di `GatewayAdapterError` in `gateway/base.py` invece che in `exceptions.py`. La scelta segue il principio di locality: l’eccezione è definita nello stesso file dell’interfaccia che la solleva, rendendo il contratto dell’ABC autosufficiente. Un consumer di `BaseGatewayAdapter` trova tutto ciò che gli serve in un unico modulo, senza dover importare da `core/exceptions.py`.

A questa si affianca una seconda scelta altrettanto deliberata: l’assenza di `ExternalToolNotFoundError`. Un tool esterno mancante non è una condizione di errore imprevista: è una condizione operativa attesa, gestita a monte durante la Phase 4 tramite `is_available()` nel registry dei connector, come descritto nella Sezione 4.4.2. Il test che dipende da un tool assente riceve uno stato `SKIP` con motivo esplicito, non un’eccezione. Introdurre

`ExternalToolNotFoundError` avrebbe aggiunto una classe senza aggiungere semantica: la condizione è già rappresentata da `TestResult(status=SKIP)`.

4.1.3 Costruzione dei Contesti di Esecuzione (D1.P3)

In Phase 3, l'engine assembla i due oggetti che attraversano l'intera pipeline: `TargetContext` e `TestContext`. La motivazione architetturale della loro separazione è discussa nella Sezione 3.2.3.

`TargetContext` è dichiarato `frozen=True` a livello di modello Pydantic: qualsiasi tentativo di scrittura dopo la costruzione produce un errore immediato nel punto in cui viene effettuato. Raccoglie tutto ciò che il sistema conosce del target prima che la pipeline inizi: l'`AttackSurface` costruita dalla specifica OpenAPI in Phase 2, le credenziali risolte dal loader, i parametri di tuning per singolo test, la configurazione dei tool esterni e l'adapter del gateway iniettato se configurato. Ogni test che legge `TargetContext` ha la garanzia assoluta che i dati che legge sono identici a quelli letti da ogni altro test nella stessa run.

`TestContext` è mutabile, ma l'accesso non avviene tramite un dizionario libero: lo stato è organizzato in tre canali tipizzati con responsabilità distinte. Il canale token gestisce le credenziali acquisite durante la run tramite `set_token/get_token/has_token`, con chiavi che sono costanti nominate (`ROLE_ADMIN`, `ROLE_USER_A`, `ROLE_USER_B`) e non stringhe arbitrarie. Il canale teardown mantiene il registro LIFO delle risorse create, descritto nella Sezione 4.3.8. Il canale dati condivisi permette a test in batch successivi di consumare risultati prodotti da batch precedenti tramite `set_shared/get_shared`, con la convenzione `"{test_id}.{nome_dato}"` che rende esplicito quale test ha prodotto ogni valore.

4.2 Discovery Dinamico e Risoluzione del Grafo

Come stabilito nella Sezione 3.2.2 e anticipato nella Fase 4 dell'elenco nella Sezione 4.1, il DAG delle dipendenze tra test non è risolto dall'engine direttamente, ma delegato a due componenti con responsabilità distinte: il `TestRegistry`, che determina a runtime quali test esistono per ispezione del filesystem, e il `DAGScheduler`, che riceve la lista dei test scoperti e calcola l'ordine topologico a partire dalle dipendenze dichiarate. Il registry opera su moduli e classi Python; lo scheduler opera su grafi di stringhe. La separazione riflette la differenza di dominio tra le due operazioni: una riguarda la composizione del set di test attivi, l'altra la loro sequenza di esecuzione.

4.2.1 Discovery Dinamico via pkgutil (D2.P1)

Il `TestRegistry` in `src/tests/registry.py` scopre le classi concrete di test a runtime attraverso tre fasi sequenziali, denominate R1, R2 e R3, senza fare ricorso ad alcun elenco di registrazione manuale:

1. **Phase R1: Scansione del package.** La scansione del package `src.tests` è affidata a `pkgutil.walk_packages()`, che ne attraversa l'intera struttura importando tutti i moduli il cui nome finale rispetta il prefisso `test_`. Il meccanismo è indipendente dall'organizzazione delle sottodirectory: qualsiasi directory di `src/tests/` configurata come package Python con il proprio `__init__.py` viene inclusa automaticamente, a prescindere dal nome. Per convenzione, i test sono organizzati in directory `domain_N/`, una per ciascun dominio di assurance, ma il registry non vincola questa struttura. Ogni file di test adotta la convenzione `test_N_M_description.py`, dove `N` è il numero di dominio, `M` il numero progressivo del test all'interno del dominio e `description` una label in `snake_case`; il `test_id` dichiarato nella `ClassVar` segue il formato puntato corrispondente `N.M`. Errori di importazione nei singoli moduli vengono catturati e registrati come `WARNING` strutturato senza interrompere la scansione degli altri.
2. **Phase R2: Estrazione delle classi concrete.** Dai moduli importati vengono estratte le sottoclassi concrete di `BaseTest` tramite `inspect.getmembers()`, scaricando la classe base stessa, le sottoclassi astratte con metodi non implementati, e le classi prive dei `ClassVar` obbligatori verificati da `has_required_metadata()`.
3. **Phase R3: Filtraggio per priorità e strategia.** Vengono trattenuti i test la cui `priority` non supera la soglia `min_priority` e la cui `strategy` è inclusa nell'insieme delle strategie abilitate in `config.yaml`. Il filtraggio avviene per classe, prima dell'istanziamento, e ogni test escluso viene registrato a livello `DEBUG` con il motivo dell'esclusione.

Il meccanismo ha una conseguenza diretta sull'estensibilità del sistema: aggiungere un test richiede la creazione di un file nella directory di dominio corretta rispettando la convenzione `test_N_M_description.py` e implementare il contratto di `BaseTest` con i `ClassVar` obbligatori; nessun altro file deve essere modificato, nessuna registrazione manuale è necessaria.

L'`ExternalTestRegistry` in `src/external_tests/registry.py` segue le stesse fasi R1-R3, adattate alla gerarchia `ExternalToolTest` e alla convenzione di nomenclatura `ext_test_*.py`, con una fase aggiuntiva denominata R4. Dopo il filtraggio, il registry raggruppa i test sopravvissuti per `tool_name` e chiama `is_available()` una sola volta per gruppo: se il tool è presente, istanzia un connector condiviso e lo inietta in tutti i test del gruppo tramite `_injected_connector`; se il tool è assente, imposta

`_skip_reason_from_registry` su ogni test del gruppo, in modo che `execute()` restituisca `SKIP` senza tentare l'invocazione. Un solo `WARNING` viene emesso per tool mancante, indipendentemente da quanti test ne dipendono.

Lo stesso principio di estensibilità vale per i test esterni: aggiungere un nuovo `ExternalToolTest` richiede la creazione di un file rispettando la nomenclatura `ext_test_*.py`, implementare il contratto di `ExternalToolTest` con i `ClassVar` obbligatori e dichiarare il `tool_name` corrispondente al connector. Questa proprietà costituisce il prerequisito tecnico del plugin system delineato come traiettoria futura nella Sezione ??: un package esterno che aggiunge la propria directory al `sys.path` e rispetta le convenzioni di nomenclatura viene rilevato automaticamente al successivo avvio della pipeline, senza modifiche al core.

4.2.2 Scheduling Topologico e DAGScheduler (D2.P2)

Il `DAGScheduler` in `src/core/dag.py` riceve la lista dei `test_id` prodotta dal registry (Sezione 4.2.1) e restituisce una sequenza ordinata di `ScheduledBatch`: ciascun batch raggruppa i test che possono essere eseguiti nello stesso livello topologico perché le loro dipendenze sono già state soddisfatte nei batch precedenti. Il meccanismo di risoluzione, introdotto a livello architetturale nella Sezione 3.2.2, si concretizza qui nel pattern `get_ready() / done()` di `graphlib.TopologicalSorter`, la libreria standard Python introdotta nella versione 3.9. Il `DAGScheduler` costruisce, a partire dal campo `depends_on` di ogni test, una mappa da ogni `test_id` all'insieme dei suoi predecessori diretti, e la passa a `TopologicalSorter` come input per la risoluzione del grafo.² Chiama poi `prepare()` per intercettare eventuali cicli prima dell'esecuzione, e drena l'ordinamento livello per livello: ogni chiamata a `get_ready()` restituisce i nodi pronti, `done(*ready_nodes)` sblocca quelli che da essi dipendono, e ogni livello produce uno `ScheduledBatch`.³

`ScheduledBatch` è un `dataclass(frozen=True)` con due campi: `batch_index` (intero che indica il livello topologico incrementando a partire da zero) e `test_ids` (lista ordinata di stringhe). L'immutabilità garantisce che l'engine non possa alterare un batch durante l'esecuzione. Il determinismo all'interno di ogni batch, requisito discusso nella Sezione 3.1.4, dipende da una singola riga: `ready_nodes = sorted(sorter.get_ready())`. `graphlib.TopologicalSorter.get_ready()` restituisce i nodi pronti senza garantire nessun ordine sull'insieme estratto: a parità di grafo e di dipendenze dichiarate, l'ordine di iterazione può variare tra run, versioni di Python o piattaforme. L'ordinamento lessicografico esplicito sul `test_id` è l'unica fonte di ordine deterministico all'interno di un

²Ogni test dichiara solo le dipendenze immediate: se il test A dipende da B e B dipende da C, è sufficiente che A dichiari `depends_on = ["B"]`. Il vincolo su C è già catturato dalla dipendenza di B su C, e `TopologicalSorter` lo propaga automaticamente nel calcolo dell'ordinamento.

³Passare un nodo a `done()` segnala al sorter che quel `test_id` è stato completato: tutti i test che lo dichiarano in `depends_on` vedono soddisfatto quel vincolo e diventano eleggibili per la successiva chiamata a `get_ready()`.

batch: due run con le stesse dipendenze dichiarate producono sempre la stessa sequenza di esecuzione, condizione necessaria perché D3.P4 sia verificabile empiricamente e non solo enunciabile.

Un caso limite, documentato nella Sezione 3.2.2 a livello architetturale, è gestito dalla stall detection: `is_active()` restituisce `True` ma `get_ready()` non produce nodi, segnalando che il sorter è in uno stato irrecuperabile nonostante l'assenza di cicli dichiarati. Il `DAGScheduler` calcola in questo caso l'insieme dei `test_id` mai estratti, li registra in un `ERROR` strutturato con il campo `unscheduled_test_ids`, e interrompe il drain. Il log fornisce all'operatore l'elenco esatto dei test coinvolti senza richiedere la lettura di un traceback generico.

4.3 Architettura dei Test Nativi e Contratto `BaseTest`

I test nativi sono le unità di verifica che comunicano direttamente con il target tramite `SecurityClient` e producono evidenza HTTP osservabile. Il loro ciclo di vita nella pipeline, dalla scoperta in Fase 4 all'invocazione in Fase 5, è descritto nella Sezione 4.1; questa sezione ne descrive la struttura interna: il contratto che ogni test deve rispettare per essere scoperto e orchestrato correttamente, le invarianti sul risultato che il sistema applica per costruzione, il meccanismo di autenticazione condivisa, le regole di tracciabilità normativa, la policy con cui le probe HTTP vengono interpretate, il meccanismo di risoluzione dei path parametrici e il catalogo dei payload di attacco, e il meccanismo di cleanup delle risorse create durante la verifica. La Milestone 1 implementa test per i domini D0, D1, D2, D3, D4, D6 e D7; il Domain-5 (Observability) è architetturalmente predisposto e pianificato per la Milestone 2.

4.3.1 Gerarchia Duale `BaseTest` / `ExternalToolTest` (D1.P4)

Il sistema ospita due categorie di test con contratti distinti: i test nativi estendono `BaseTest` in `src/tests/base.py`; i test con tool esterni estendono `ExternalToolTest` in `src/external_tests/base.py`. Le due classi base sono gerarchie parallele e indipendenti, per ragioni tecniche precise che la struttura del sistema rende evidenti.

La scelta di mantenere le due gerarchie separate risponde a un problema concreto di ereditarietà indesiderata. `BaseTest` espone metodi come `_log_transaction()` e `_probe_endpoint()`, che operano tramite `SecurityClient` per eseguire richieste HTTP contro il target applicativo. Un `ExternalToolTest` non usa `SecurityClient`: delega la raccolta di dati al proprio connector, che invoca un processo di sistema o una libreria Python. Fare ereditare `ExternalToolTest` da `BaseTest` avrebbe reso disponibile a ogni

test esterno un'interfaccia pensata per un pattern di integrazione diverso, ampliando la superficie di errore e oscurando il contratto che il test deve effettivamente rispettare.

Dal punto di vista dell'engine, la distinzione è quasi invisibile: entrambe le gerarchie producono un `TestResult`, vengono scoperte dal registry con gli stessi meccanismi di Phase R1-R3 descritti nella Sezione 4.2.1 (con l'aggiunta di Phase R4 per i test esterni), e vengono schedulate dal `DAGScheduler`. L'unica differenza visibile in `engine.py` è un `isinstance` check al momento dell'invocazione, che seleziona la firma corretta di `execute()`. Il campo `TestResult.source` con tipo `Literal["native", "external"]` propaga la distinzione fino al report, dove i risultati nativi e quelli di tool specializzati vengono riportati con sezioni visivamente distinte all'interno dello stesso dominio.

Il contratto che `BaseTest` impone a ogni implementazione concreta si articola in due livelli:

- **Dichiarazione statica via `ClassVar`:** ogni test deve dichiarare `test_id` (identificatore univoco corrispondente all'ID nella metodologia), `domain` (dominio di assurance 0-7), `priority` (P0-P3), `strategy` (`BLACK_BOX`, `GREY_BOX`, `WHITE_BOX`), `depends_on` (lista di `test_id` predecessori per il DAG), `cwe_id` e `tags` (riferimenti normativi). `has_required_metadata()` scarta le classi con `ClassVar` incompleti prima che vengano istanziate, così le implementazioni parziali non raggiungono mai la pipeline.
- **Implementazione di `execute(target, ctx, store)`:** il metodo che garantisce le due invarianti discusse nella Sezione 4.1.1: restituire sempre un `TestResult` e non propagare mai eccezioni verso l'engine.

4.3.2 Invarianti di `TestResult` a Livello di Tipo (D4.P9)

Ogni test produce un `TestResult`, il modello Pydantic che trasporta lo stato finale della verifica verso il report. La coerenza tra lo stato di un `TestResult` e il suo contenuto, se affidata alla disciplina distribuita di implementazioni indipendenti, è una proprietà che il sistema non può garantire strutturalmente: uno stato `FAIL` senza evidenza, uno `SKIP` senza motivazione, un `PASS` con findings allegati sono tutti costruibili sintatticamente senza che il sistema se ne accorga, producendo entry di report formalmente validi ma privi di significato verificabile. La soluzione adottata sposta l'enforcement a livello di tipo, tramite un `@model_validator(mode="after")` in `src/core/models/results.py` che applica tre invarianti nel momento in cui ogni test costruisce il proprio risultato, ovvero alla chiamata di `_make_pass()`, `_make_fail()` o `_make_skip()`, prima che l'oggetto raggiunga qualsiasi consumer:

1. `status=FAIL` richiede `findings` non vuoto. Un FAIL senza evidenza non è un FAIL: è un'asserzione.
2. `status=SKIP` richiede `skip_reason` non `None`. Ogni SKIP deve essere documentabile e auditabile.
3. `status=PASS` richiede `findings` vuoto. Un PASS non può coesistere con violazioni dichiarate.

La centralizzazione in un singolo `model_validator` ha un effetto architetturale preciso: il contratto tra stato e contenuto del `TestResult` è enforced nel punto in cui l'oggetto viene costruito, indipendentemente da quale test lo costruisce e da quale helper utilizza. Una violazione solleva `pydantic.ValidationError` nel punto in cui l'helper `_make_pass()`, `_make_fail()` o `_make_skip()` costruisce l'oggetto, con un traceback che punta direttamente alla riga incriminata nel test responsabile. Ogni nuova implementazione di test ottiene la verifica automaticamente: il validator intercetta risultati inconsistenti nel punto stesso della costruzione, senza che il test debba contenere logica di validazione propria.

4.3.3 Auth Abstraction Layer e Idempotenza dei Token (D2.P5)

I test `GREY_BOX` non possono iniziare la propria probe senza token validi: il gateway respinge qualsiasi richiesta priva di credenziali prima che la logica di autorizzazione venga anche solo valutata. Con una parte significativa dei 18 test attivi in Milestone 1 in modalità `GREY_BOX`, distribuire la logica di login in ogni test produrrebbe autenticazioni ripetute verso lo stesso target per ruoli già acquisiti. `acquire_tokens()` in `src/tests/helpers/auth.py` risolve questo problema centralizzando la gestione dei token nel `TestContext`: alla chiamata, la funzione verifica prima se il `TestContext` contiene già un token valido per il ruolo richiesto tramite `ctx.has_token(role)`; solo se il token è assente esegue il login HTTP e lo scrive nel canale token tramite `ctx.set_token(role, token)`. Il numero di login effettivi verso il target è così esattamente uguale al numero di ruoli distinti configurati, indipendentemente da quanti test li richiedano.

Il dispatcher in `auth.py` legge il campo `auth_type` dal `config.yaml` e delega l'acquisizione dei token all'implementazione corrispondente, astraendo i test dal dettaglio del protocollo. Nella Milestone 1 sono supportati due metodi: `forgejo_token`, che acquisisce un token tramite l'endpoint Forgejo-specifico `POST /api/v1/users/{username}/tokens` con HTTP Basic Auth, e `jwt_login`, un flusso generico configurabile tramite `login_endpoint`, `username_body_field` e `token_response_path`, adatto a qualsiasi target che esponga un endpoint di login JSON, da crAPI a Django REST Framework. Aggiungere il supporto per un nuovo meccanismo richiede una nuova implementazione del flusso e un ramo

aggiuntivo nel dispatcher; i test rimangono invariati perché il loro contratto con il layer di autenticazione è espresso esclusivamente in termini di ruolo richiesto.

4.3.4 Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)

Ogni test porta con sé, come `ClassVar`, i riferimenti normativi della garanzia che verifica: `cwe_id` per la classe di vulnerabilità nel Common Weakness Enumeration (CWE), e `tags` per le categorie Open Web Application Security Project (OWASP), i riferimenti National Institute of Standards and Technology (NIST) e i documenti tecnici Request for Comments (RFC). Entrambi vengono propagati fino al modello `Finding` tramite il campo `references: list[str]` e riportati dal report HTML a fianco di ogni risultato: un analista può risalire direttamente dal finding al codice CWE, alla categoria OWASP API Security e alle sezioni normative pertinenti senza consultare documentazione esterna.

La tracciabilità presuppone una distinzione semantica precisa tra due tipi di annotazione che un test può produrre. Un `Finding` è evidenza di una violazione: appartiene esclusivamente ai risultati `FAIL`, il suo contenuto deve essere sufficiente a giustificare la valutazione negativa, e viene conteggiato nei totali aggregati del report. Un `InfoNote` è un'annotazione informativa allegabile a un risultato `PASS`: documenta contesto rilevante per l'analista senza alterare il verdetto né influenzare i contatori. La distinzione è architetturale: i contatori del report aggregano esclusivamente oggetti `Finding`, rendendo il totale affidabile e non influenzabile da osservazioni di contorno. La coerenza tra tipo di risultato e tipo di annotazione è enforced come descritto nella Sezione 4.3.2: il `model_validator` di `TestResult` garantisce che un `PASS` non possa coesistere con `Finding` nel momento stesso della costruzione dell'oggetto.

Il valore operativo della distinzione emerge con chiarezza nel test 4.3, che verifica la presenza di un meccanismo di circuit breaking sull'infrastruttura del gateway. Kong OSS, nella configurazione di Milestone 1, non implementa circuit breaking attivo ma espone healthcheck passivi che svolgono una funzione parzialmente equivalente. Il test registra `PASS` perché il requisito minimo è soddisfatto, e allega una `InfoNote` che documenta la differenza tra il compensating control osservato e il circuit breaking completo. L'analista ottiene un quadro preciso senza che il sistema produca né un falso `FAIL` né un `PASS` silenzioso che nasconde la sfumatura.

4.3.5 Oracle a Tre Esiti e Safe HTTP Probing Policy (D4.P7)

Ogni test nativo che interroga un endpoint del target riceve una risposta HTTP che il tool deve interpretare e tradurre in un verdetto preciso. Il problema non è banale:

uno status **404 Not Found** su un endpoint parametrico come `/users/{id}/repos` può significare che il gateway ha rifiutato la richiesta per mancanza di autenticazione, che il path non esiste per il valore di `{id}` usato nel probe, o che l'endpoint è genuinamente non protetto ma il path era errato. Interpretare un **404** come **FAIL** in tutti i casi genera falsi positivi inaccettabili in un assessment che si propone come affidabile.

La policy adottata definisce un framework a tre esiti applicato a ogni probe HTTP nei test nativi: ogni risposta è classificata come **ENFORCED** (il controllo funziona), **BYPASSED** (il controllo è assente o mal configurato) o **INCONCLUSIVE** (il risultato non è interpretabile con certezza sufficiente per produrre un **Finding**). Il mapping specifico degli status code a ciascun esito dipende dalla natura del test: per i test **BLACK_BOX** che verificano l'enforcement dell'autenticazione, la forma è la seguente:

1. **ENFORCED**: la risposta è **401** o **403**. Il gateway ha applicato un controllo di accesso sul chiamante non autenticato.
2. **BYPASSED**: la risposta è **2xx**. Il gateway non ha richiesto autenticazione. Il controllo è assente o mal configurato.
3. **INCONCLUSIVE**: qualsiasi altro status. Il risultato non è interpretabile con certezza sufficiente per produrre un **Finding** ⁴.

Solo un esito **BYPASSED** produce un **Finding** e contribuisce a un risultato **FAIL**. Un esito **INCONCLUSIVE** viene registrato nell'evidenza con il suo status effettivo, ma non altera il verdetto del test.

La codifica a tre esiti presuppone che la risposta ricevuta rifletta il comportamento del controllo di accesso, non un effetto collaterale della probe stessa. Questa condizione regge per i path fissi della specifica, dove il test raggiunge una risorsa che esiste e la risposta è interpretabile. Per i path parametrici la situazione è più sottile: un endpoint come `/users/{id}/repos` ha una struttura valida, ma il valore concreto del parametro determina se quella specifica risorsa esiste sul target. Con un valore che non corrisponde a nessun oggetto reale, il gateway o il backend restituisce **404** perché la risorsa è assente, non perché stia applicando un controllo di accesso; il risultato osservato è indistinguibile da un corretto rifiuto di autenticazione. Classificare quel **404** come **INCONCLUSIVE** è l'unica risposta metodologicamente corretta. Da questa asimmetria nasce la segregazione in due tier: il Tier A raccoglie i path fissi e quelli parametrici per cui il seed è configurato (Sezione 4.3.6), dove la probe usa valori concreti verificati sul target e la risposta è interpretabile; il Tier B raccoglie i path parametrici senza seed, dove il risultato

⁴Esempio osservato su Forgejo: senza path seed, il probe su `/api/v1/repos/{owner}/{repo}` usa il placeholder "1" per entrambi i parametri; Forgejo risponde **404** perché la risorsa non esiste, la richiesta non raggiunge il middleware di autenticazione e il risultato è **INCONCLUSIVE**. Con seed configurato, la stessa richiesta produce **ENFORCED** o **BYPASSED**.

è **INCONCLUSIVE** e l'evidenza segnala all'operatore che configurare il seed risolverebbe l'ambiguità.

Un caso limite merita attenzione esplicita. Le probe con metodo **DELETE** su endpoint parametrici presentano un rischio operativo: se il gateway è mal configurato e non applica autenticazione, una richiesta **DELETE** con un path valido potrebbe eliminare risorse reali sul target. La policy prevede un fallback sicuro: quando il path seed non è configurato per un endpoint **DELETE** parametrico, il probe viene eseguito con il valore letterale "**apiguard-probe**" come sostituto del parametro. Un path con quel valore non corrisponde a nessuna risorsa reale su alcun sistema, garantendo che il probe non causi effetti collaterali indesiderati anche in presenza di un gateway non autenticato.

4.3.6 Path Seed System (D2.P7)

Il path seed system affronta una tensione strutturale dell'agnosticismo applicativo. Derivare tutta la conoscenza del target dalla specifica OpenAPI significa conoscere la topologia degli endpoint ma non i valori concreti dei loro parametri: `/users/{owner}/repos/{repo}` è un endpoint documentato, ma senza sapere quale **owner** e quale **repo** esistano realmente sul target, la probe usa un placeholder il cui path quasi certamente non esiste, producendo **404** e un risultato **INCONCLUSIVE**. Il seed risolve questa tensione fornendo per ogni parametro la possibilità di specificare un valore concreto verificato sul target⁵: la generazione tramite il comando CLI **generate-seed**, che interroga la specifica OpenAPI, produce un template da completare e inserire in **config.yaml**. Con il seed configurato per Forgejo, i path con `{owner}` e `{repo}` producono risposte interpretabili: **ENFORCED** se il gateway applica il controllo atteso, **BYPASSED** se non lo applica. La Sezione ?? riporta i dati quantitativi di questo effetto sull'assessment di Milestone 1.

4.3.7 Catalogo dei Payload di Attacco (D2.P8)

I test nativi applicano il principio di separazione tra dati e logica di valutazione che, come verrà illustrato nella Sezione 4.4.1, governa anche il contratto dei connector: i dati di input alla verifica risiedono separati dalla logica che li interpreta. Nel caso dei test nativi, questo si concretizza nel catalogo dei payload. Le sequenze di URL usate dal test **SSRF**, gli header malformati per i test di input validation, e altri vettori di attacco risiedono in moduli puri in **src/tests/data/**, distinti dalla logica che li applica. Aggiungere un nuovo vettore richiede la modifica del catalogo, senza toccare il test che lo utilizza. La separazione ha una conseguenza operativa concreta in contesti enterprise, dove i payload di attacco sono soggetti a un processo di approvazione separato dal codice applicativo.

⁵Un valore verificato è un identificatore di una risorsa realmente esistente nel deployment; nel caso di Forgejo: un utente, un repository, e così via.

4.3.8 Teardown Best-Effort in Ordine LIFO (D4.P6)

I test che raggiungono la logica applicativa del target spesso creano risorse necessarie alla verifica: token di autenticazione temporanei, utenti di test, risorse effimere. Lasciare queste risorse sul target al termine dell'assessment viola la proprietà di non-interferenza: un assessment successivo potrebbe incontrare uno stato alterato, e il target produttivo potrebbe accumulare risorse orfane nel tempo.

Il cleanup delle risorse si basa sulla registrazione esplicita al momento della creazione: ogni volta che un test crea una risorsa persistente, chiama immediatamente `ctx.register_resource_for_tpath, headers)` passando l'endpoint di cancellazione. La coppia creazione-registrazione avviene nella stessa transazione logica; se il test va in eccezione tra le due operazioni, la risorsa potrebbe non essere rimossa. Questa condizione è documentata come limite noto: il meccanismo di isolamento degli errori descritto nella Sezione 4.1.1 non può recuperare uno stato che non è mai stato registrato.

In Phase 6, l'engine chiama `ctx.drain_resources()`, che restituisce le risorse in ordine LIFO. Tale ordine inverso rispetto alla creazione è necessario per gestire correttamente le dipendenze tra risorse: se un test ha creato prima un repository e poi una issue al suo interno, la issue dipende dal repository; cancellarli nell'ordine di creazione eliminerebbe prima il repository, rendendo impossibile la cancellazione della issue. Invertendo l'ordine, la issue viene rimossa prima che il repository smetta di esistere. Per ogni risorsa restituita, l'engine tenta la chiamata HTTP di cancellazione tramite `SecurityClient`; un fallimento solleva `TeardownError`, catturato e registrato come `WARNING` strutturato. Il fallimento del teardown è un avvertimento operativo, non una condizione che invalida i risultati prodotti nelle fasi precedenti. La Sezione ?? riporta i dati dell'audit di Milestone 1, che ha rilevato zero risorse non rimosse su due run indipendenti.

4.4 Gerarchia dei Connettori e Integrazione Tool Esterni

La controparte della gerarchia duale introdotta nella Sezione 4.3.1 è `ExternalToolTest`: i test che delegano la raccolta di dati a tool specializzati invece di comunicare direttamente con il target tramite `SecurityClient`. Seguono lo stesso ciclo di vita nella pipeline (scoperti in Fase 4 dall'`ExternalTestRegistry` con l'aggiunta di Phase R4, invocati in Fase 5), ma con un pattern di integrazione strutturalmente diverso. Scanner di vulnerabilità, analizzatori TLS, fuzzer producono tipi di evidenza che un client HTTP generico non può raccogliere: la negoziazione TLS richiede dialogo a livello di protocollo, l'analisi con template nuclei richiede una base di signature aggiornata. Il layer dei connector è il punto in cui questa delega prende forma concreta: isola la logica di invocazione di ogni tool, normalizza il suo output in un modello comune e restituisce al test un oggetto strutturato invece del raw output del processo. In assenza di questo layer, ogni test

esterno conterrebbe sia la logica di verifica della garanzia sia la meccanica di subprocess management o di importazione della libreria, accoppiando due responsabilità che evolvono per ragioni distinte.

4.4.1 Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)

La gerarchia dei connector si articola su tre livelli, ciascuno dei quali eredita solo i meccanismi pertinenti al proprio pattern di integrazione.

`BaseConnector` in `src/connectors/base.py` è il primo livello: un Abstract Base Class (ABC) puro con un solo `ClassVar` (`TOOL_NAME: str`) e tre metodi astratti che costituiscono il contratto universale verso il test. `is_available()` verifica se il tool è utilizzabile nell'ambiente corrente; `get_version()` restituisce la versione del tool per fini di audit; `run(target_url, timeout_seconds)` esegue il tool e restituisce un `ConnectorResult`. Il parametro `timeout_seconds` è privo di valore di default per design: ogni chiamante deve fornirlo esplicitamente dal `config.yaml`, rendendo impossibile dimenticarlo.

Il secondo livello si biforca a seconda della natura del tool:

- `BaseSubprocessConnector` serve i tool invocati come processo di sistema (`NucleiConnector`, `TestsslConnector`): fornisce implementazioni concrete di `is_available()` e `get_version()` basate su subprocess, e il metodo `_run_subprocess()` che gestisce esecuzione, timeout e parsing dell'output. La ricerca del binario avviene su tre canali, fermandosi alla primo soddisfatto, nel seguente ordine: binario locale nella directory `tools/` del progetto, poi `shutil.which()` sul PATH di sistema, infine una variabile d'ambiente per i deployment in cui il tool è esposto come microservizio HTTP. Il medesimo connector funziona così senza modifiche in sviluppo locale, in CI/CD e in ambienti containerizzati.
- `BaseLibraryConnector` serve i tool Python-native (`SslyzeConnector`): usa `importlib.util.find_spec()` per verificare la disponibilità della libreria senza invocare alcun subprocess, e legge la versione tramite `importlib.metadata`. La disponibilità è determinata dalla presenza del package nell'ambiente Python, non da un binario sul filesystem.

Le implementazioni concrete si limitano a dichiarare le `ClassVar` specifiche del tool⁶ e implementare `run()`, il cui contratto formalizza una separazione precisa tra raccolta di dati e valutazione della garanzia. Il connector restituisce un `ConnectorResult`, modello Pydantic frozen che trasporta `tool_name`, `tool_version`, `raw_output: dict[str, Any]`,

⁶Per `BaseSubprocessConnector`: `TOOL_NAME`, `BINARY_NAME`, `LOCAL_TOOLS_SUBDIR` e `SERVICE_ENV_VAR`.
Per `BaseLibraryConnector`: `TOOL_NAME` e `LIBRARY_NAME`.

`exit_code`, `execution_time_ms` e `timed_out`: dati grezzi, privi di qualsiasi giudizio sulla sicurezza del target.

La valutazione è responsabilità esclusiva dell'`ExternalToolTest`, che implementa `_evaluate()` e applica il proprio oracle al `ConnectorResult`. La ragione di questa separazione è la riusabilità: il `NucleiConnector` non sa nulla della garanzia che il test vuole verificare, e per questo può essere condiviso da test con logiche di valutazione completamente diverse. Un secondo test che vuole usare nuclei per injection scanning scriverebbe un `_evaluate()` diverso ma userebbe lo stesso connector, senza alcuna modifica alla logica di invocazione.

4.4.2 Lifecycle dei Connector e Dependency Injection (D2.P4)

Come descritto nella Sezione 4.2.1, la Phase R4 dell'`ExternalTestRegistry` raggruppa i test per `tool_name` e chiama `is_available()` una sola volta per gruppo. Il risultato di quella chiamata determina il lifecycle del connector per tutti i test del gruppo: disponibilità o assenza del tool si traduce in due comportamenti distinti, gestiti tramite due attributi di istanza di `ExternalToolTest`.

Se il tool è disponibile, il registry istanzia un connector condiviso e lo scrive in `_injected_connector` su ogni test del gruppo: tutti condividono la stessa istanza, costruita una volta sola. Se il tool è assente, il registry scrive invece in `_skip_reason_from_registry` la motivazione di skip già formulata. Al momento dell'esecuzione, `_run()` controlla questo campo come prima operazione: se valorizzato, restituisce `SKIP` immediatamente senza tentare la costruzione del connector né interrogare il filesystem. In entrambi i casi, la decisione è presa una volta sola in Phase R4 e propagata a tutti i test del gruppo tramite questi due attributi.

La conseguenza operativa misurabile è nel log di esecuzione. Con cinque test che dipendono da nuclei e il binario assente, il log produce un solo `WARNING` del tipo *"nuclei not found: 5 tests will SKIP"*, invece di cinque entry identiche. In un assessment con molti tool e molti test, questa differenza rende il log leggibile invece che ridondante. Il comportamento complessivo del sistema in presenza di infrastruttura parzialmente disponibile, compresi gli scenari con Admin API assente o master switch disattivato, è descritto nella Sezione 4.5.2.

4.4.3 Classificazione A/B dei Connector (D2.P6)

I connector sono classificati in due categorie che riflettono il loro rapporto con la copertura nativa. I connector di Categoria A coprono garanzie che i test nativi non possono raggiungere con soli probe HTTP: la loro assenza produce `SKIP` su verifiche altrimenti inac-

cessibili. I connector di Categoria B estendono la copertura su garanzie già verificate parzialmente dai test nativi, aggiungendo profondità senza essere necessari per l'assessment di base. Nella Milestone 1, `NucleiConnector` e `TestsslConnector/SslyzeConnector` appartengono entrambi alla Categoria A: il primo perché l'analisi di Shadow API tramite template specializzati non è replicabile con probe HTTP generici; i secondi perché la negoziazione TLS richiede dialogo a livello di protocollo che supera le capacità di un client HTTP standard.

4.4.4 Isolamento della Dipendenza AGPL (D7.P2)

`sslyze`, la libreria Python per l'analisi TLS, è distribuita sotto licenza GNU Affero General Public License (AGPL) v3. Le dipendenze dichiarate in `[project.dependencies]` di `pyproject.toml` vengono installate automaticamente con qualsiasi `pip install apiguard-assurance`; includervi `sslyze` propagherebbe i suoi obblighi copyleft a chiunque distribuisca o utilizzi il tool, indipendentemente dall'uso effettivo della libreria. La sezione `[project.optional-dependencies]` del `pyproject.toml` è pensata esattamente per questo tipo di situazione: raccoglie le dipendenze con vincoli di licenza o peso computazionale che non devono essere imposti per default, lasciando all'operatore la scelta consapevole di installarle tramite un extra esplicito.

`sslyze` è dichiarata in `[project.optional-dependencies.sslyze]`. L'installazione esplicita tramite `pip install "apiguard-assurance[sslyze]"` abilita `ext.1.5.sslyze` e comporta l'accettazione consapevole della licenza AGPL per quella componente. In assenza dell'installazione, `SslyzeConnector.is_available()` restituisce `False` tramite `importlib.util.find_spec()`, il test riceve `SKIP` con motivazione esplicita, e il core rimane privo di qualsiasi import condizionale della libreria, isolandola strutturalmente. Il pattern è replicabile per qualsiasi dipendenza con vincoli analoghi configurando il `pyproject.toml` appositamente.

4.5 Implementazione del Gateway Adapter

La Sezione 3.2.4 ha stabilito il principio di agnosticismo verso il gateway e introdotto `BaseGatewayAdapter` come interfaccia astratta di sola lettura verso il piano di configurazione amministrativo. Questa sezione ne descrive la concretizzazione: l'implementazione Kong che viene iniettata nel `TargetContext` in Fase 3 e i tre livelli di degradazione controllata che il sistema applica quando parti dell'infrastruttura non sono disponibili.

4.5.1 KongGatewayAdapter: Accesso Read-Only all'Admin API

KongGatewayAdapter in `src/core/gateway/kong.py` è l'unica implementazione concreta di BaseGatewayAdapter nella Milestone 1, e realizza il principio dichiarato nella Sezione 3.2.4: i test WHITE_BOX 3.3, 4.2, 4.3 e 6.4 chiamano `target.gateway.get_plugins()`, `get_services()` e `get_upstreams()` tramite l'interfaccia astratta, senza contenere nessun riferimento a Kong. L'implementazione interroga la Admin API di Kong DB-less tramite richieste HTTP GET. Una scelta implementativa merita particolare attenzione: il componente usa `httpx` direttamente invece di `SecurityClient`. Le chiamate alla Admin API sono audit di configurazione dell'infrastruttura, non traffico di test verso il target applicativo; appartengono a un confine di fiducia distinto, non devono apparire nell'EvidenceStore e non sono soggette alla policy di retry del `SecurityClient`. Un client `httpx` dedicato mantiene i due confini separati nel codice in modo esplicito e verificabile.

I metodi pubblici dell'adapter, `get_routes()`, `get_plugins()`, `get_services()`, `get_upstreams()` e `get_plugin_by_name()`, delegano tutti la lettura a `_fetch_paginated()`. La Admin API di Kong restituisce le collezioni in formato paginato: ogni risposta contiene un campo "data" con gli elementi della pagina corrente e un campo "next" con il path della pagina successiva, oppure `null` se non ci sono ulteriori elementi. `_fetch_paginated()` segue questo cursore iterativamente finché "next" è `null`, concatenando i risultati in un'unica lista. In modalità DB-less tutti gli oggetti entrano in una singola pagina e "next" è sempre `null`, quindi il ciclo termina alla prima iterazione; lo stesso meccanismo funziona correttamente in ambienti con Kong con database, dove le risorse possono essere distribuite su più pagine.

Il ciclo di vita dell'adapter si articola su due momenti distinti. In Fase 3, `check_connectivity()` verifica che la Admin API sia raggiungibile prima di popolare `TargetContext.gateway`: se la verifica fallisce, il campo rimane `None` e tutti i test WHITE_BOX transitano a SKIP per l'intera run. Durante l'esecuzione dei test (Fase 5), qualsiasi errore nelle chiamate HTTP verso l'Admin API viene incapsulato in `GatewayAdapterError`, che il test cattura e converte in `TestResult(ERROR)` senza propagare l'eccezione all'engine.

4.5.2 Degradazione Controllata a Tre Livelli (D4.P5)

Un assessment eseguito in ambienti eterogenei incontra sistematicamente situazioni in cui parti dell'infrastruttura non sono disponibili: tool esterni non installati, Admin API non esposta, integrazione con tool esterni disabilitata per policy. La risposta del sistema non è un errore a cascata, ma una degradazione controllata su tre livelli distinti che produce risultati parziali corretti con motivazioni esplicite in ogni caso.

- **Livello 1 - master switch globale.** Il flag `external_tools.enabled` in `config.yaml` disabilita l'intera gerarchia `ExternalToolTest` in un'unica operazione: l'`ExternalTestRegistry` restituisce una lista vuota senza scandagliare il filesystem né importare moduli, con zero overhead e zero SKIP nel log. Serve per ambienti in cui la presenza di tool esterni è impossibile per policy di sicurezza o di deployment.
- **Livello 2 - tool esterno assente.** Quando il master switch è attivo ma un singolo tool non è disponibile, la Phase R4 descritta nella Sezione 4.2.1 imposta `_skip_reason_from_registry` su ogni test del gruppo: ogni test riceve SKIP con motivazione esplicita, e un solo WARNING viene emesso nel log per tool mancante.
- **Livello 3 - Admin API del gateway assente.** Se `gateway_adapter` non è configurato in `config.yaml` oppure se `check_connectivity()` fallisce in Phase 3, `TargetContext.gateway` rimane `None`. Ogni test `WHITE_BOX` chiama `_requires_admin_api(target)` come prima operazione e restituisce SKIP con testo esplicito senza eseguire alcuna logica di verifica. L'assessment copre in questo scenario i soli domini `BLACK_BOX` e `GREY_BOX`, con un numero di SKIP pari al numero di test `WHITE_BOX` nel set attivo.

In tutti e tre i casi, gli SKIP prodotti sono distinguibili nel report per motivazione (campo `skip_reason`), permettendo all'operatore di separare le garanzie non verificate per impossibilità tecnica da quelle non verificate per prerequisiti mancanti.

4.6 Evidence Store e Sanitizzazione delle Credenziali

Ogni verifica di sicurezza produce due tipi di output: un verdetto, espresso come `TestResult`, e l'evidenza che lo sostiene, composta dalle transazioni HTTP effettuate dai test nativi e dai dati grezzi prodotti dai tool esterni. L'evidenza è ciò che distingue un risultato verificabile da un'asserzione: senza di essa, un FAIL non può essere riprodotto né contestato. Raccoglierla introduce però una responsabilità: un audit trail non sanitizzato che contiene token o credenziali è esso stesso un rischio di sicurezza: il file viene condiviso con il team di analisi e archiviato nei sistemi di ticketing, dove la sua esposizione sfugge al controllo di chi ha eseguito l'assessment. Questa sezione descrive come `EvidenceStore` gestisce raccolta affidabile e sanitizzazione strutturale durante la Fase 5, e come i tre deliverable finali vengono prodotti in Fase 7, come anticipato nella Sezione 4.1: `evidence.json` (archivio forense), `assessment_report.html` (report operativo) e `apiguard_report.json` (output strutturato per integrazione con sistemi CI/CD).

4.6.1 Ciclo di Vita dello Streaming Evidence Store (D4.P1)

Il design dell'`EvidenceStore` è motivato nella Sezione 3.2.5, dove viene discusso il problema strutturale del buffer a capacità fissa che produceva audit trail rotti in silenzio. Questa sezione descrive l'implementazione concreta che ha risolto quel problema.

Il ciclo di vita dell'`EvidenceStore` si articola in quattro operazioni sincronizzate con le fasi dell'engine. In Phase 3, l'engine istanzia `EvidenceStore` e crea la directory temporanea `outputs/evidence_tmp/`. Prima di eseguire ogni test in Phase 5, l'engine chiama `store.begin_test(test_id)`: il metodo apre il file `outputs/evidence_tmp/{test_id}.jsonl` in modalità `append` e lo mantiene aperto per l'intera durata del test. Durante l'esecuzione, ogni chiamata a `store.log_transaction()` o `store.pin_artifact()` scrive immediatamente un oggetto JavaScript Object Notation (JSON) su una singola riga del file con `flush()` esplicito: zero buffering in memoria, zero rischio di perdita in caso di crash. Al termine del test, `store.end_test()` chiude il file. In Phase 7, `store.merge_and_finalize()` legge tutti i file `.jsonl` dalla directory temporanea, li ordina per `test_id` lessicografico per garantire un output deterministico, li concatena in un unico array JSON e li scrive nel file definitivo `outputs/evidence.json`. La directory temporanea viene poi rimossa.

Riportando la soluzione iniziale con `deque(maxlen=100)`, il primo record di un test con più di 100 transazioni veniva silenziosamente scartato dalla coda, generando `Finding.evidence_ref` che puntavano a record inesistenti nell'audit trail senza produrre nessun errore visibile. Il design corrente elimina questa classe di bug per costruzione: ogni record scritto su disco è permanente, e il numero di record per test è illimitato. L'unico parametro che incide sulla dimensione dell'archivio è `RESPONSE_BODY_MAX_CHARS = 10 000`: la costante tronca il body di risposta di ogni singolo record per evitare che risposte molto grandi saturino il disco, lasciando il conteggio dei record invariato.

La proprietà di recuperabilità è un effetto collaterale dell'architettura. Se il processo va in crash tra Phase 5 e Phase 7, i file `.jsonl` rimangono su disco e sono leggibili manualmente con qualsiasi strumento JSON Lines (JSONL)-compatibile: l'evidenza parziale è recuperabile anche in assenza di un run completo.

4.6.2 Sanitizzazione Centralizzata delle Credenziali (D4.P2)

Un audit trail che contiene token di autenticazione, API key o password è esso stesso una superficie di rischio: `evidence.json` viene condiviso con il team di sicurezza, archiviato nei sistemi di ticketing, e potenzialmente incluso in repository. Delegare la redazione ai singoli connector presuppone che ogni implementazione, incluse quelle di terze parti, ricordi di sanitizzare i propri campi sensibili: una convenzione distribuita su un sistema estensibile non è verificabile staticamente e non regge nel tempo.

La responsabilità è centralizzata in un unico punto, con due meccanismi distinti per i due tipi di evidenza. Gli artifact dei tool esterni passano per `EvidenceStore._sanitize_artifact()`, invocato automaticamente da `pin_artifact()` prima della scrittura su disco: il connector non è tenuto a fare nulla, la sanitizzazione avviene per costruzione su qualsiasi output grezzo prodotto da un connector. Per le transazioni HTTP dei test nativi, la redazione opera al livello del modello: il `field_validator` su `EvidenceRecord.request_headers` sostituisce sempre il valore dell'header `Authorization` con `[REDACTED]` al momento della costruzione dell'oggetto, indipendentemente da come il record viene creato e prima che raggiunga l'`EvidenceStore`.

La sanitizzazione degli artifact applica tre meccanismi in sequenza, sovrapposti per coprire pattern che i singoli meccanismi da soli non intercetterebbero:

1. **Key-pattern matching:** una regex controlla ricorsivamente ogni chiave del dizionario rispetto a 11 pattern sensibili (`token`, `password`, `api_key`, `apikey`, `authorization`, `bearer`, `secret`, `credential`, `auth`, `private_key`, `access_token`), confrontando nomi interi e non sottostringhe. Qualsiasi chiave che corrisponde vede il proprio valore sostituito con `[REDACTED]`, indipendentemente dal tipo.
2. **JWT value detection:** la regex `_SANITIZE_JWT_PATTERN` riconosce la struttura a tre segmenti base64url separati da punto e redacta qualsiasi stringa con quella forma, indipendentemente dalla chiave che la contiene. Cattura i token che appaiono come valori di chiavi con nomi arbitrari.
3. **Header prefix matching:** qualsiasi stringa che inizia con `"Bearer "`, `"Basic "`, `"Token "` o `"token "` viene redactata, intercettando i token che compaiono nei valori di header HTTP inclusi nei record di evidenza.

La sovrapposizione dei tre meccanismi copre scenari che nessuno dei tre individualmente intercetterebbe: un token JWT in una chiave `"result"` arbitraria supera il primo meccanismo (nome non riconosciuto) ma viene catturato dal secondo (struttura base64url); un header `Authorization` in un dict annidato viene catturato dal primo meccanismo (key-pattern matching ricorsivo). Un connector di terze parti che non sanitizzi i propri dati non introduce una violazione nell'audit trail: la garanzia è nel punto di scrittura su disco, non nel punto di produzione del dato.

I tre meccanismi appena descritti si applicano esclusivamente agli artifact dei connector. La sanitizzazione delle transazioni HTTP prodotte dai test nativi avviene su un percorso distinto, già descritto all'inizio di questa sezione: il `field_validator` su `EvidenceRecord.request_headers` garantisce la redazione dell'header `Authorization` al momento della costruzione del modello, prima che il record raggiunga l'`EvidenceStore`.

4.6.3 Dual Audit Trail e Report Domain-Centric (D5.P5)

I tre deliverable finali dell'assessment assolvono funzioni distinte e si rivolgono a interlocutori diversi. La distinzione più importante è quella tra i due meccanismi di audit trail, che operano con politiche di raccolta diverse.

`evidence.json` è l'archivio forense. Contiene due categorie di record unite in un'unica struttura serializzata: le transazioni HTTP che i test nativi hanno marcato come evidenza di FAIL, con il body di risposta fino a 10.000 caratteri, e gli artifact dei tool esterni, ossia gli output JSON dei connector sanitizzati e pinned tramite `pin_artifact()`. Ogni record è autosufficiente per ricostruire la probe o l'analisi che ha rivelato la vulnerabilità. Il file è strutturato come oggetto JSON con campi di intestazione (`generated_at_utc`, `record_count`) che accompagnano l'array dei record, in modo che un sistema esterno possa leggere metadati e dati dallo stesso file. Il formato è direttamente ingestibile da sistemi di log aggregation.

`assessment_report.html` è il deliverable operativo, generato da `report/renderer.py` tramite Jinja2. Include l'executive summary con le statistiche aggregate, la tabella dei risultati per test con riferimenti normativi, e per ogni test il `TransactionSummary`: il log completo di ogni transazione HTTP eseguita, incluse quelle sui test con esito ENFORCED o PASS, con body troncati a 2.000 caratteri per richiesta e 1.000 per risposta. Il `TransactionSummary` ha funzione di proof of coverage: permette di verificare che il tool abbia effettivamente interrogato gli endpoint dichiarati e di fare triage senza aprire `evidence.json`. Il report è un file HyperText Markup Language (HTML) completamente autonomo: tutto il Cascading Style Sheets (CSS) e il JavaScript sono inline nel template, senza dipendenze da Content Delivery Networks (CDNs) o asset esterni. Può essere aperto da qualsiasi browser offline, allegato a un ticket, inviato per email o archiviato come artefatto di un'esecuzione CI/CD, senza che il destinatario debba installare nulla. Per gestire report di grandi dimensioni, l'audit trail di ogni test usa lazy loading e rimane collassato per default.

`apiguard_report.json` è la serializzazione strutturata del `ReportData`: contiene tutti i risultati inclusi PASS e SKIP, le statistiche aggregate e i metadati del run, in formato adatto all'integrazione con pipeline CI/CD, dashboard e sistemi Security Information and Event Management (SIEM).

La struttura del report è determinata dall'aggregazione prodotta da `report/builder.py`: per ogni dominio di assurance, i risultati dei test nativi precedono quelli dei test con tool specializzati, distinzione che emerge dal campo `TestResult.source` senza richiedere sezioni fisicamente separate nel template.

Bibliografia

- [1] “OWASP Top Ten Web Application Security Risks | OWASP Foundation.” [Online].
Available: <https://owasp.org/www-project-top-ten/>